



Treehouse tHYPE Audit Report

May 18, 2026





Table of Contents

Summary	2
Overview	3
Issues	4
[WP-L1] Router's <code>ValidatorNotActive</code> custom error is unreachable because Kinetiq's <code>validatorActiveState</code> reverts on unknown validators	4
[WP-L2] The <code>KinetiqRedelegate.executeAction()</code> return value and emitted amount may differ from the actual redelegated amount.	7
[WP-G3] Unnecessary copying of <code>calldata</code> to <code>memory</code> wastes gas.	11
[WP-N5] <code>TreehouseRouter.constructor()</code> fails to emit a <code>ValidatorUpdated</code> event when updating the <code>validator</code> .	13
Appendix	15
Disclaimer	16

Summary

This report has been prepared for Treehouse smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



Overview

Project Summary

Project Name	Treehouse
Codebase	https://github.com/treehouse-gaia/tHYPE-protocol
Commit	c852631f185b9749fa0e98f8bf5ff3a57c3c15d3
Language	Solidity

Audit Summary

Delivery Date	May 18, 2026
Audit Methodology	Static Analysis, Manual Review
Total Issues	4

[WP-L1] Router's `ValidatorNotActive` custom error is unreachable because Kinetiq's `validatorActiveState` reverts on unknown validators

Low

Issue Description

The new `IValidatorManager.validatorActiveState(address)` interface is declared to return a `bool`, and the Router's `_checkValidatorActive` branches on that boolean to throw its own `ValidatorNotActive(address)` custom error:

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity =0.8.24;
3
4  /// @dev Kinetiq ValidatorManager proxy on Hyperliquid.
5  ///     Only includes the view functions used by tHYPE.
6  interface IValidatorManager {
7      /// @notice Returns true if the validator is currently in the active set.
8      function validatorActiveState(address validator) external view returns (bool);
9  }
```

In practice, the deployed Kinetiq `ValidatorManager` proxy at `0x4b797a93dfc3d18cf98b7322a2b142fa8007508f` does **not** return `false` for unknown validators — it **reverts** with the string `"Validator does not exist"`. Verified on-chain:

```

$ crpc call 999 0x4b797a93dfc3d18cf98b7322a2b142fa8007508f \
    "validatorActiveState(address)(bool)" 0x0000000000000000000000000000000001
Error: execution reverted: Validator does not exist
```

Consequence: on every path that calls `_checkValidatorActive` — the Router's constructor, `setValidator`, and the `KinetiqRedelegate` action — when a non-existent or unregistered validator address is passed, the transaction reverts with Kinetiq's opaque string revert rather than the Router's intended `ValidatorNotActive(_validator)` custom error. The `if (!...)` branch is effectively dead code for the "unknown validator" case and only fires for validators

that Kinetiq *registered* and then *deactivated*.

This is purely a UX / observability issue. Liveness is unaffected (the check still fails closed), and funds are never at risk. But monitoring tooling / front-ends that key off the `ValidatorNotActive(address)` selector will not see it trigger on the most common misconfiguration (a typo / stale address at deployment or `setValidator` time).

```

174    /// @dev Reverts unless `_validator` is in the Kinetiq active validator set.
175    ///     Resolves the ValidatorManager proxy from the StakingManager so the
176    ///     Router cannot drift out of sync with Kinetiq's source of truth.
177    function _checkValidatorActive(address _stakingManager, address _validator)
        internal view {
178        address vm_ = IStakingManager(_stakingManager).validatorManager();
179        if (!IValidatorManager(vm_).validatorActiveState(_validator)) {
180            revert ValidatorNotActive(_validator);
181        }
182    }

```

Call sites that rely on the custom error surfacing:

```

73    depositCapInHype = _depositCapInHype;
74    if (_validator == address(0)) revert InvalidValidator();
75    _checkValidatorActive(_stakingManager, _validator);
76    validator = _validator;

```

```

149    function setValidator(address _newValidator) external onlyOwner {
150        if (_newValidator == address(0)) revert InvalidValidator();
151        _checkValidatorActive(STAKINGMANAGER, _newValidator);
152        emit ValidatorUpdated(_newValidator, validator);
153        validator = _newValidator;
154    }

```

```

60
61    // Pre-flight: toValidator must be in Kinetiq's active set. Kinetiq's
62    // StakingFacet performs the same check, but guarding here gives a clear
63    // revert reason inside the action chain instead of a generic facet error.
64    address vm_ = IStakingManager(stakingManager).validatorManager();

```

```

65     if (!IValidatorManager(vm_).validatorActiveState(_toValidator)) {
66         revert ValidatorNotActive(_toValidator);
67     }

```

Recommendation

Wrap the external view in a `try/catch` inside `_checkValidatorActive` so that both the "unknown validator" (revert) and "registered but inactive" (returns `false`) cases surface as the Router's own `ValidatorNotActive(_validator)` custom error. Mirror the same pattern in `KinetiqRedelegate` .

```

function _checkValidatorActive(address _stakingManager, address _validator) internal
view {
    address vm_ = IStakingManager(_stakingManager).validatorManager();
    try IValidatorManager(vm_).validatorActiveState(_validator) returns (bool active) {
        if (!active) revert ValidatorNotActive(_validator);
    } catch {
        revert ValidatorNotActive(_validator);
    }
}

```

Additional gas cost is ~200 units per call; the check runs only at deployment, `setValidator` , and `KinetiqRedelegate` — not per deposit — so the overhead is negligible.

Status

✓ Fixed

[WP-L2] The `KinetiqRedelegate.executeAction()` return value and emitted amount may differ from the actual redelegated amount.

Low

Issue Description

Per the `stakingManager.redelegate()` implementation, it appears that the amount used for L1 operations is truncated to 8 decimals.

However, `KinetiqRedelegate.executeAction()` uses the original `params.kHYPERAmount` (18 decimals, which may not be divisible by `1e10`) as the `amountRedelegated`. This likely creates a discrepancy with the `_convertTo8Decimals(hypeAmount, false)` used within `stakingManager.redelegate()`.

Note: The `StakingFacet` below is source from <https://hyperevmscan.io/address/0x393d0b87ed38fc779fd9611144ae649ba6082109#readProxyContract#F9> (`stakingManager` is `0x393d0b87ed38fc779fd9611144ae649ba6082109`, and `bytes4(keccak256("redelegate(address,address,uint256)))` is `0x6bd8f804`).

HyperEvmScan/StakingFacet/src/facets/StakingFacet.sol
<https://vscode.blockscan.com/999/0x4F2A7FDAB14dF9800B2115ff702ecbB9D7E8097d>

```

181     /**
182      * @notice Request redelegation of stake from one validator to another
183      * @dev Expresses a routed-stake preference for the off-chain operator bot to
184      * fulfill
185      * @param fromValidator Validator to move stake from (address(0) resolves to
186      * protocol default)
187      * @param toValidator Validator to move stake to (address(0) resolves to
188      * protocol default, must be active)
189      * @param kHYPERAmount Amount of kHYPER representing the stake to redelegate
190      */
191     function redelegate(address fromValidator, address toValidator, uint256
    kHYPERAmount) external nonReentrant {
192         _checkNotPaused();
193         _checkWhitelist(msg.sender);
194     }

```



```

192     // Resolve address(0) to protocol default validator
193     address delegation = validatorManager.getDelegation(address(this));
194     if (fromValidator == address(0)) fromValidator = delegation;
195     if (toValidator == address(0)) toValidator = delegation;
196
197     if (fromValidator == toValidator) revert InvalidAddress();
198     if (kHYPEAmount == 0) revert InvalidAmount();
199     if (!validatorManager.validatorActiveState(toValidator)) revert
ValidatorNotActive();
200     if (tokenAddress.balanceOf(msg.sender) < kHYPEAmount) revert
InsufficientTokenBalance();
201
202     uint256 hypeAmount = stakingAccountant.kHYPEToHYPE(kHYPEAmount);
203     uint256 truncatedAmount = _convertTo8Decimals(hypeAmount, false);
204
205     emit RedelegationRequested(msg.sender, fromValidator, toValidator,
kHYPEAmount, hypeAmount, truncatedAmount);
206 }

```

HyperEvmScan/StakingFacet/src/StakingManagerStorage.sol

<https://vscode.blockscan.com/999/0x4F2A7FDAB14dF9800B2115ff702ecbB9D7E8097d>

```

161     /**
162     * @notice Converts amount from 18 decimals to 8 decimals for L1 operations
163     * @param amount Amount in 18 decimals
164     * @param roundUp Whether to round up (for withdrawals) or down (for deposits)
165     * @return truncatedAmount Amount in 8 decimals
166     */
167     function _convertTo8Decimals(uint256 amount, bool roundUp) internal pure
returns (uint256 truncatedAmount) {
168         truncatedAmount = amount / 1e10;
169
170         // For withdrawals, round up to ensure users get at least the requested
amount
171         if (roundUp && amount % 1e10 > 0) {
172             truncatedAmount += 1;
173         }
174
175         // Add check for uint64 overflow
176         if (truncatedAmount > type(uint64).max) revert AmountExceedsUint64();
177

```

```

178         return truncatedAmount;
179     }

```

```

    @@ 1,8 @@

9
10  /// @title Redelegate kHYPE stake between validators
11  contract KinetiqRedelegate is ActionBase, KinetiqHelper {
    @@ 12,29 @@

30
31  /// @inheritdoc ActionBase
32  function executeAction(
33      bytes calldata _callData,
34      uint8[] memory _paramMapping,
35      bytes32[] memory _returnValues
36  ) public payable virtual override returns (bytes32) {
37      Params memory params = parseInputs(_callData);
38      params.fromValidator = _parseParamAddr(params.fromValidator, _paramMapping[0],
    _returnValues);
39      params.toValidator = _parseParamAddr(params.toValidator, _paramMapping[1],
    _returnValues);
40      params.kHYPEAmount = _parseParamUint(params.kHYPEAmount, _paramMapping[2],
    _returnValues);
41
42      (uint amountRedelegated, bytes memory logData) =
    _kinetiqRedelegate(params.fromValidator, params.toValidator, params.kHYPEAmount);
43      emit ActionEvent(NAME, logData);
44
45      return bytes32(amountRedelegated);
46  }
47
48  ////////////////////////////////// ACTION LOGIC //////////////////////////////////
49
50  /// @notice Redelegate kHYPE stake from one validator to another
51  /// @param _fromValidator Source validator address
52  /// @param _toValidator Target validator address
53  /// @param _kHYPEAmount Amount of kHYPE to redelegate
54  function _kinetiqRedelegate(
55      address _fromValidator,
56      address _toValidator,
57      uint _kHYPEAmount

```



```

58     ) internal returns (uint amountRedelegated, bytes memory logData) {
59         amountRedelegated = _kHYPEAmount;
60
61         // Pre-flight: toValidator must be in Kinetiq's active set. Kinetiq's
62         // StakingFacet performs the same check, but guarding here gives a clear
63         // revert reason inside the action chain instead of a generic facet error.
64         address vm_ = IStakingManager(stakingManager).validatorManager();
65         if (!IValidatorManager(vm_).validatorActiveState(_toValidator)) {
66             revert ValidatorNotActive(_toValidator);
67         }
68
69         IStakingManager(stakingManager).redelegate(_fromValidator, _toValidator,
70             _kHYPEAmount);
71         logData = abi.encode(_fromValidator, _toValidator, _kHYPEAmount);
72     }
73
74     function parseInputs(bytes memory _callData) internal pure returns (Params
75     memory inputData) {
76         inputData = abi.decode(_callData, (Params));
77     }

```

Status

✓ Fixed

[WP-G3] Unnecessary copying of `calldata` to `memory` wastes gas.

Gas

Issue Description

<https://docs.soliditylang.org/en/v0.8.34/types.html#reference-types>

An assignment or type conversion that changes the data location will always incur an automatic copy operation

<https://docs.soliditylang.org/en/v0.8.34/types.html#data-location>

If you can, try to use `calldata` as data location because it will avoid copies and also makes sure that the data cannot be modified. Arrays and structs with `calldata` data location can also be returned from functions, but it is not possible to allocate such types.

```

32     function executeAction(
33         bytes calldata _callData,
34         uint8[] memory _paramMapping,
35         bytes32[] memory _returnValues
36     ) public payable virtual override returns (bytes32) {
37         Params memory params = parseInputs(_callData);
38         params.fromValidator = _parseParamAddr(params.fromValidator, _paramMapping[0],
39             _returnValues);
39         params.toValidator = _parseParamAddr(params.toValidator, _paramMapping[1],
40             _returnValues);
40         params.kHYPERAmount = _parseParamUint(params.kHYPERAmount, _paramMapping[2],
41             _returnValues);
42         (uint amountRedelegated, bytes memory logData) =
43             _kinetiqRedelegate(params.fromValidator, params.toValidator, params.kHYPERAmount);
43         emit ActionEvent(NAME, logData);
44
45         return bytes32(amountRedelegated);
46     }

```

<https://github.com/treehouse-gaia/tHYPE-protocol/blob/197f15d6747eddd154a312b9e8480d77dd72d619/src/strategy/actions/kinetiq/KinetiqRedelegate.sol#L74-L76>

```
74     function parseInputs(bytes memory _callData) internal pure returns (Params
    memory inputData) {
75         inputData = abi.decode(_callData, (Params));
76     }
```

Recommendation

Consider changing the data location of the `parseInputs(bytes memory _callData)` parameter to `calldata` .

Note: This issue is not limited to the `KinetiqRedelegate` action; other actions (including the `KinetiqStake` action modified in this PR) exhibit similar patterns.

Status

✓ Fixed

[WP-N5] `TreehouseRouter.constructor()` fails to emit a `ValidatorUpdated` event when updating the `validator` .

Issue Description

For reference, #C0_L152 correctly performs `emit ValidatorUpdated(_newValidator, validator);`

<https://github.com/treehouse-gaia/tHYPE-protocol/blob/197f15d6747eddd154a312b9e8480d77dd72d619/src/TreehouseRouter.sol#L54-L79>

```

54     constructor(
    @@ 55,57 @@
58         address _stakingManager,
    @@ 59,61 @@
62         address _validator
63     ) Ownable(_creator) {
    @@ 64,65 @@
66         STAKINGMANAGER = _stakingManager;
    @@ 67,73 @@
74         if (_validator == address(0)) revert InvalidValidator();
75         _checkValidatorActive(_stakingManager, _validator);
76         validator = _validator;
77
78         IERC20(IAU).approve(address(TASSET), type(uint).max);
79     }

```

```

145     /**
146      * @notice sets validator address
147      * @param _newValidator new validator address
148      */
149     function setValidator(address _newValidator) external onlyOwner {
150         if (_newValidator == address(0)) revert InvalidValidator();
151         _checkValidatorActive(STAKINGMANAGER, _newValidator);
152         emit ValidatorUpdated(_newValidator, validator);
153         validator = _newValidator;
154     }

```



Status

✓ Fixed



Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.